Report

v. 1.0

Customer

Uniswap

Smart Contract Audit

TWAP Auction

14th November 2025

Report prepared by

ABDK Consulting

Contents

1 Changelog

#	Date	Author	Description
0.1	14.11.25	A. Zveryanskaya	Initial Draft
0.2	14.11.25	A. Zveryanskaya	Minor revision
1.0	14.11.25	A. Zveryanskaya	Release

2 Summary

This document presents the results of a smart contract audit performed by ABDK Consulting for Uniswap, specifically for their `TWAP Auction` system. The audit was conducted between 20th October and 14th November 2025 by Mikhail Vladimirov and Dmitry Khovratovich. The primary objective of this audit was to assess the security, correctness, and efficiency of the TWAP auction mechanism, with particular focus on the pricing logic, bid processing, checkpoint management, and token distribution mechanisms.

Our conclusion is that the system demonstrates a reasonable level of maturity, with well-structured contract architecture and comprehensive functionality. The codebase leverages established third-party libraries from Solady and Uniswap v4-core, indicating adherence to industry best practices. However, several critical areas required attention to ensure the system operates as intended under all conditions.

Our examination revealed two major issues that required immediate attention. The first major finding CVF-1 identified a flaw in the `Auction.sol` contract where setting the clearing price to the minimum clearing price could result in incorrect calculation of the demand above the clearing price. This issue was particularly concerning as it affects the core auction mechanics. The second major issue CVF-2 found in `CheckpointStorage.sol` revealed unclear behavior where there was no validation to ensure that new checkpoint block numbers are higher than the last checkpointed block, potentially leading to checkpoint overwrites or non-monotonic ordering. Both issues have been addressed by the development team.

Beyond these major findings, we identified several moderate issues related to scalability concerns with loop iterations that could potentially exceed block gas limits, suboptimal number format conversions throughout the codebase, and various efficiency improvements in the fixed-point arithmetic implementations. Overall, with the major issues resolved the system demonstrates solid engineering and is approaching production readiness.

It is important to note that a security audit is not a guarantee of absolute security but rather a snapshot of the system's security posture at a specific point in time. While we strive to uncover all potential issues, the evolving nature of blockchain technology and DeFi means new vulnerabilities can emerge.

3 System overview

This section provides a high-level overview of the `TWAP Auction` system developed by Uniswap. The System implements a time-weighted average price auction mechanism designed to facilitate fair and efficient token distribution. The primary purpose is to enable projects to conduct token sales through a gradual price discovery process that spans multiple blocks, allowing participants to submit bids at various price points while the system dynamically calculates a clearing price based on supply and demand dynamics.

We were asked to review:

- Original Code
- Code with Fixes

TWAPAuction files:

/			
	TokenCurrency Storage.sol	TickStorage.sol	PermitSingle Forwarder.sol
	CheckpointStorage.sol	BidStorage.sol	AuctionStepStorage.sol
	AuctionFactory.sol	Auction.sol	
libraries/			
	ValueX7Lib.sol	ValidationHookLib.sol	FixedPoint96.sol
	FixedPoint128.sol	CurrencyLibrary.sol	ConstantsLib.sol
	CheckpointLib.sol	BidLib.sol	AuctionStepLib.sol
interfaces/			
	IValidationHook.sol	ITokenCurrency Storage.sol	ITickStorage.sol
	IPermitSingle Forwarder.sol	ICheckpointStorage.sol	IBidStorage.sol
	IAuctionStepStorage.sol	IAuctionFactory.sol	IAuction.sol
interfaces/external/			
	IERC20Minimal.sol	IDistributionStrategy.sol	IDistributionContract.sol

The System is implemented entirely in Solidity and leverages well-established development practices for Ethereum smart contracts. The codebase follows a modular

architecture with clear separation of concerns between core logic, storage management, and external interfaces. The System integrates several third-party libraries from Solady, including fixed-point mathematical operations, safe type casting utilities, and safe transfer mechanisms. Additionally, the System utilizes SSTORE2 for efficient off-chain data storage and retrieval, which is particularly relevant for storing packed auction step configurations.

At the architectural level, the System consists of several interconnected components that work together to manage the auction lifecycle. The `Auction` contract serves as the central orchestrator, coordinating bid submissions, price calculations, and token distributions. The `AuctionFactory` contract provides a standardized deployment mechanism, allowing for consistent creation of new auction instances with customizable parameters. This factory pattern ensures that each auction maintains its own isolated state while sharing common implementation logic.

The System employs a sophisticated storage architecture divided into specialized contracts. The `BidStorage` contract maintains records of all participant bids, tracking amounts, prices, and ownership information. The `CheckpointStorage` contract captures the auction state at specific block heights, enabling historical lookups and settlement calculations. The `TickStorage` contract organizes price levels into discrete ticks, similar to order book structures, allowing efficient traversal of bid prices. The `AuctionStepStorage` contract manages the temporal progression of the auction through predefined steps, each specifying the quantity of tokens to be sold per block. Finally, the `TokenCurrencyStorage` contract handles the dual-asset nature of the auction, managing both the token being distributed and the currency being raised, whether native ether or standard tokens.

The System implements extensive mathematical operations through specialized libraries. The fixed-point arithmetic libraries provide precision-safe calculations using 96-bit and 128-bit scaling factors, which are essential for accurately computing token allocations and currency requirements without losing precision to integer division. The `ValueX7` library introduces a custom scaling factor based on milli-bips, representing one ten-millionth as a precision unit for percentage calculations. This unconventional denominator provides fine-grained control over auction parameters while maintaining computational efficiency.

The auction mechanism itself operates through a multi-block process where tokens are gradually released according to a predetermined schedule. Participants submit bids specifying their maximum acceptable price and desired token quantity. As the auction progresses block by block, the System continuously recalculates a clearing price based on the cumulative demand at various price levels compared to the available token supply. The clearing price represents the threshold at which supply meets demand, and all bids at or above this price receive proportional allocations.

Integration with external systems occurs through several well-defined interfaces. The System connects to the Permit2 protocol through the `PermitSingleForwarder` contract, enabling gasless approvals where users can authorize token transfers through signed messages rather than separate transactions. The validation hook interface allows auction creators to implement custom logic that executes before each bid is accepted, enabling use cases such as allowlist enforcement, contribution limits, or other participation criteria. The distribution strategy interface provides flexibility in how tokens

are ultimately distributed after the auction concludes, whether through immediate transfers, vesting schedules, or other custom mechanisms.

The System handles both native ether and standard tokens as the currency for raising funds. The `CurrencyLibrary` provides a unified abstraction over these two asset types, treating the zero address as a special case representing native ether while other addresses represent token contracts. This design allows auction creators to configure their preferred fundraising currency without requiring separate contract implementations.

Participants interact with the System primarily through the bid submission functions exposed by the `Auction` contract. After tokens are received and the auction begins, users can submit bids specifying their maximum price and desired amount. The System validates these bids, updates internal data structures, and recalculates the clearing price as necessary. Once the auction concludes and all blocks are processed, participants can claim their allocated tokens or retrieve refunds for any unfilled portions of their bids. The System ensures that all settlements are calculated fairly based on the final clearing price and the chronological order of bid submissions.

The System architecture prioritizes gas efficiency, deterministic behavior, and mathematical precision. The use of packed data structures, optimized storage patterns, and carefully designed iteration loops reflects a focus on minimizing transaction costs while maintaining the complex logic required for fair price discovery. The checkpoint mechanism provides transparency by permanently recording auction state at key moments, allowing participants to verify their allocations and understand how the clearing price evolved over time.

4 Methodology

The methodology is not a strict formal procedure, but rather a selection of methods and tactics combined differently and tuned for each particular project, depending on the project structure and technologies used, as well as on client expectations from the audit.

- **General Code Assessment**. The code is reviewed for clarity, consistency, style, and for whether it follows best code practices applicable to the particular programming language used. We check indentation, naming convention, commented code blocks, code duplication, confusing names, confusing, irrelevant, or missing comments etc. At this phase we also understand overall code structure.
- **Entity Usage Analysis**. Usages of various entities defined in the code are analysed. This includes both: internal usages from other parts of the code as well as potential external usages. We check that entities are defined in proper places as well as their visibility scopes and access levels are relevant. At this phase, we understand overall system architecture and how different parts of the code are related to each other.
- **Access Control Analysis**. For those entities, that could be accessed externally, access control measures are analysed. We check that access control is relevant and done properly. At this phase, we understand user roles and permissions, as well as what assets the system ought to protect.
- **Code Logic Analysis**. The code logic of particular functions is analysed for correctness and efficiency. We check whether the code actually does what it is supposed to do, whether the algorithms are optimal and correct, and whether proper data types are used. We also make sure that external libraries used in the code are up to date and relevant to the tasks they solve in the code. At this phase we also understand data structures used and the purposes they are used for.

We classify issues by the following severity levels:

- **Critical issue** directly affects the smart contract functionality and may cause a significant loss.
- **Major issue** is either a solid performance problem or a sign of misuse: a slight code modification or environment change may lead to loss of funds or data. Sometimes it is an abuse of unclear code behaviour which should be double checked.
- **Moderate issue** is not an immediate problem, but rather suboptimal performance in edge cases, an obviously bad code practice, or a situation where the code is correct only in certain business flows.
- **Recommendations** cover code style, best practices and general improvements.

5 Our findings

We found 2 major, and a few less important issues. All identified Major issues have been fixed.

Fixed 2 out of 2 issues

6 Major Issues

CVF-1 FIXED

- **Category** Flaw
- **Source** Auction.sol

Description Setting clearing price to the minimum clearing price could make the calculated demand above clearing price incorrect.

Recommendation Properly update the demand above the clearing price, or clearly explain why it is fine not to update it.

Client Comment *Fix with inline comment.*

```
291 if (updateStateVariables) {
        $sumCurrencyDemandAboveClearingQ96 =
            ↪ sumCurrencyDemandAboveClearingQ96_;
```

```
299 if (clearingPrice < minimumClearingPrice) {
300     clearingPrice = minimumClearingPrice;
```

CVF-2 FIXED

- **Category** Unclear behavior
- **Source** CheckpointStorage.sol

Description There is no check to ensure the provided block number is higher than the last checkpointed block. This could lead to a situation when the last checkpoint is overwritten or when checkpoint blocks doesn't go in acesnding order.

Recommendation Add an explicit check to ensure the new checkpoint block number is higher then the last checkpoint block number.

Client Comment *This doesn't necessarily break anything but there is an assumption that checkpoints should be monotonic so this should be fixed. The cost of adding a guard is ~52 gas on first write and ~130 thereafter.*

```
52 $_checkpoints[blockNumber] = checkpoint;
$lastCheckpointedBlock = blockNumber;
```

7 Moderate Issues

CVF-3 INFO

- **Category** Flaw
- **Source** AuctionStepLib.sol

Description The length of “data” is ignored, so memory outside of “data” could be read here.

Recommendation Add a proper length check.

Client Comment *Won’t fix because this is checked by caller.*

```
23 let packedValue := mload(add(add(data, 0x20), offset))
```

CVF-4 INFO

- **Category** Suboptimal
- **Source** Auction.sol

Description These loops don’t scale and could exceed the block gas limit.

Recommendation Refactor the code to process blocks in logarithmic time or allow processing pending blocks in multiple transactions.

Client Comment *Won’t fix because it is not feasible to cause this iteration to revert with OOG, and we don’t believe that it is worth a refactoring for preformance reasons at this point in time.*

```
230 while (blockNumber > end) {
```

```
270 while (
        // Loop while the currency amount above the clearing price is
            ↪ greater than the required currency at `
            ↪ nextActiveTickPrice_`
        (
            nextActiveTickPrice_ != MAX_TICK_PTR
                && sumCurrencyDemandAboveClearingQ96_ >= TOTAL_SUPPLY *
                    ↪ nextActiveTickPrice_
        )
        // If the demand above clearing rounds up to the `
            ↪ nextActiveTickPrice`, we need to keep iterating over ticks
        // This ensures that the `nextActiveTickPrice` is always the
            ↪ next initialized tick strictly above the clearing price
        || clearingPrice == nextActiveTickPrice_
    ) {
```

CVF-5 INFO

- **Category** Suboptimal
- **Source** IDistributionContract.sol

Recommendation It would be more efficient to pass distribution details as arguments to the function, so the contract won't need to figure them out on its own, querying for token balances and doing other expensive stuff.

```
16 function onTokensReceived() external;
```

CVF-6 FIXED

- **Category** Suboptimal
- **Source** ConstantsLib.sol

Description This limit appears quite low. Many high-quality DeFi protocols allow token amounts up to 2^256-1, but de-facto standard is to at least support amounts as high as 2^128-1. Here the maximum amount is 10 million times smaller.

Recommendation Refactor the code to at least support amounts as high as 2^128-1.

Client Comment *This constant is unused and removed.*

```
11 /// @notice The maximum allowable amount for currency or token
   ↪ related values
   uint128 constant MAX_AMOUNT = type(uint128).max / 1e7;
```

CVF-7 INFO

- **Category** Unclear behavior
- **Source** BidLib.sol

Description It is unclear why amount has to be scaled, as token amounts usually already have decimals.

Recommendation Use unscaled amount.

Client Comment *Need to scale to prevent wei level precision loss. By multiplying by Q96 we lose precision in the fractional bits.*

```
16 uint256 amountQ96; // User's demand
```

CVF-8 INFO

- **Category** Unclear behavior
- **Source** CurrencyLibrary.sol

Description The original error message from the failed transfer is dropped here.

Recommendation Return the original error message to the caller.

Client Comment *Won't fix, this was previously audited as a part of v4-core.*

```
34 if (!success) {
       revert NativeTransferFailed();
```

```
65 if (!success) {
       revert ERC20TransferFailed();
```

CVF-9 INFO

- **Category** Suboptimal
- **Source** CurrencyLibrary.sol

Description Clearing free memory is redundant, as Solidity tolerates garbage in free memory.

Recommendation Don't clear free memory.

Client Comment *Won't fix, this was previously audited as a part of v4-core.*

```
59 // Now clean the memory we used
60 mstore(fmp, 0) // 4 byte `selector` and 28 bytes of `to` were stored
      ↪  here
   mstore(add(fmp, 0x20), 0) // 4 bytes of `to` and 28 bytes of `amount
      ↪ ` were stored here
   mstore(add(fmp, 0x40), 0) // 4 bytes of `amount` were stored here
```

CVF-10 FIXED

- **Category** Suboptimal
- **Source** ValidationHookLib.sol

Recommendation This could be rewritten using a try...catch(bytes) block.

```
24 (bool success, bytes memory reason) = address(hook).call(
       abi.encodeWithSelector(IValidationHook.validate.selector,
          ↪ maxPrice, amount, owner, sender, hookData)
   );
   if (!success) revert ValidationHookCallFailed(reason);
```

CVF-11 INFO

- **Category** Bad naming
- **Source** AuctionStepLib.sol

Description The name is confusing, as it gives no clue this value is per block.
Recommendation Rename to “mpsPerBlock”.
Client Comment *Won’t fix, too big of a refactor.*

```
5 uint24 mps; // Mps to sell per block in the step
```

CVF-12 INFO

- **Category** Suboptimal
- **Source** IDistributionContract.sol

Recommendation This function should take the received amount as an argument to save on the “balanceOf” call and also to distinguish legitimate tokens from tokens that ended up on the contract’s balance by mistake.

```
8 function onTokensReceived() external;
```

CVF-13 FIXED

- **Category** Suboptimal
- **Source** IAuctionStepStorage.sol

Description Indexing block number parameters seems odd, as indexes only support strict equality.
Recommendation Don’t index block number parameters or clearly explain the reasonng.

```
39 event AuctionStepRecorded(uint256 indexed startBlock, uint256
   ↪ indexed endBlock, uint24 mps);
```

CVF-14 INFO

- **Category** Suboptimal
- **Source** AuctionStepStorage.sol

Description This read the data back from the deployed contract just to validate it.
Recommendation Validate data before writing. Trust the SSTORE2 library to revert in case data wasn’t written successfully.
Client Comment *Fine with the gas overhead from reading it back from SSTORE2 as its only done on deployment. Feels much more explicit to validate that it was stored correctly.*

```
38 _validate(_pointer);
```

CVF-15 INFO

- **Category** Suboptimal
- **Source** TokenCurrencyStorage.sol

Description Wrapping token as a currency is inefficient (due to zero address check in transfer) and also is not strictly correct, as zero token address will be interpreted as plain ether.

Recommendation Use plain IERC20 transfer call.

Client Comment *Won't fix.*

```
76 Currency.wrap(address(TOKEN)).transfer(TOKENS_RECIPIENT, amount);
```

CVF-16 INFO

- **Category** Suboptimal
- **Source** Auction.sol

Description This check doesn't allow receiving tokens in several separate transfers.

Recommendation Allow receiving less than TOTAL_SUPPLY, but don't set the "$_tokensReceived" flag in such a case.

Client Comment *It does support this, there is no requirement that 'onTokensReceived' must be called after each ERC20 transfer. This is called manually.*

```
118 if (TOKEN.balanceOf(address(this)) < TOTAL_SUPPLY) {
        revert InvalidTokenAmountReceived();
```

CVF-17 FIXED

- **Category** Suboptimal
- **Source** Auction.sol

Description This function uses a mixture of number formats and constantly converts between them. This makes code less efficient, more error prone and less readable.

Recommendation This function uses a mixture of number formats and constantly converts between them. This makes code less efficient, more error prone and less readable.

```
153 function _sellTokensAtClearingPrice(Checkpoint memory _checkpoint,
    ↪ uint24 deltaMps)
```

CVF-18 FIXED

- **Category** Documentation
- **Source** Auction.sol

Description The semantics of the “prevTickPrice” argument is unclear.

Recommendation Explain in the documentation comment and/or rename the argument to “prevTickPriceHint”.

Client Comment *Fixed via documentation.*

```
347 function _submitBid(uint256 maxPrice, uint128 amount, address owner,
      ↪  uint256 prevTickPrice, bytes calldata hookData)
```

```
411 function submitBid(uint256 maxPrice, uint128 amount, address owner,
      ↪ uint256 prevTickPrice, bytes calldata hookData)
```

CVF-19 FIXED

- **Category** Procedural
- **Source** CheckpointStorage.sol

Description These functions contain too much business logic for a single storage contract.

Recommendation Move them away from storage contract into a business-logic contract or library.

Client Comment *Valid, business logic should not live here.*

```
64 function _accountFullyFilledCheckpoints(Checkpoint memory upper,
     ↪ Checkpoint memory startCheckpoint, Bid memory bid)
```

```
82 function _accountPartiallyFilledCheckpoints(
```

```
110 function _calculateFill(Bid memory bid, uint256
      ↪ cumulativeMpsPerPriceDelta, uint24 cumulativeMpsDelta)
```

CVF-20 INFO

- **Category** Procedural
- **Source** CheckpointStorage.sol

Description Such a complicated logic is a result of using multiple number formats and a hybrid fixed-point format, where the denominator is 2^96 * 10^7, i.e. neither a power of 2 no a power of 10.

Recommendation Stick to a single number format, such as WAD or X96.

Client Comment *Won't fix.*

```
89 // tickDemandQ96 is a summation of bid effective amounts, so we must
      ↪  scale up the bid
90 // by 1e7 and divide by `mpsRemainingInAuctionAfterSubmission` such
      ↪ that we can
   // apply the ratio of the bid demand to the tick demand to the
      ↪ currencyRaisedAtClearingPriceQ96_X7
```

```
121 // The tokens filled from the bid are calculated from its effective
       ↪ amount, not the raw amount in the Bid struct
    // As such, we need to multiply it by 1e7 and divide by `
       ↪ mpsRemainingInAuctionAfterSubmission`.
    // We also know that `cumulativeMpsPerPriceDelta` is over `mps`
       ↪ terms, and has not bee divided by 100% (1e7) yet.
    // Thus, we can cancel out the 1e7 terms and just divide by `
       ↪ mpsRemainingInAuctionAfterSubmission`.
```

8 Recommendations

CVF-21 INFO

- **Category** Bad datatype
- **Source** IDistributionContract.sol

Recommendation The parameter type should be “IERC20”.

```
8 error InvalidToken(address token);
```

CVF-22 INFO

- **Category** Bad datatype
- **Source** IDistributionStrategy.sol

Recommendation The type for the “distributionContract” parameter should be “IDistributionContract”.

```
13 event DistributionInitialized(address indexed distributionContract,
   ↪ address indexed token, uint256 totalSupply);
```

CVF-23 INFO

- **Category** Bad datatype
- **Source** IDistributionStrategy.sol

Recommendation The type for the “token” parameter should be “IERC20”.

```
13 event DistributionInitialized(address indexed distributionContract,
   ↪ address indexed token, uint256 totalSupply);
```

CVF-24 INFO

- **Category** Bad datatype
- **Source** IDistributionStrategy.sol

Recommendation Events are usually named via nouns, such as “Distribution”.

```
13 event DistributionInitialized(address indexed distributionContract,
   ↪ address indexed token, uint256 totalSupply);
```

CVF-25 INFO

- **Category** Bad datatype
- **Source** IDistributionStrategy.sol

Recommendation The type for the “token” argument should be “IERC20”.

```
25 function initializeDistribution(address token, uint256 totalSupply,
   ↪ bytes calldata configData, bytes32 salt)
```

CVF-26 INFO

- **Category** Procedural
- **Source** ValueX7Lib.sol

Description We didn’t review this file.

```
5 import {FixedPointMathLib} from 'solady/utils/FixedPointMathLib.sol'
   ↪ ;
```

CVF-27 FIXED

- **Category** Suboptimal
- **Source** ValueX7Lib.sol

Description Usage of “divUint256” here is misleading, as the result is actually not a X7 value.

Recommendation Rewrite as: return ValueX7.unwrap(value)/X7;

```
70 return ValueX7.unwrap(value.divUint256(X7));
```

CVF-28 FIXED

- **Category** Documentation
- **Source** ValueX7Lib.sol

Description This note is confusing.

Recommendation Elaborate more or remove.

```
74 /// @dev Ensure that `b` and `c` should be compared against the
   ↪ ValueX7 value
```

```
80 /// @dev Ensure that `b` and `c` should be compared against the
   ↪ ValueX7 value
```

CVF-29 INFO

- **Category** Procedural
- **Source** CheckpointLib.sol

Description We didn't review this file.

```
9 import {FixedPointMathLib} from 'solady/utils/FixedPointMathLib.sol'
    ↪ ;
```

CVF-30 INFO

- **Category** Procedural
- **Source** CheckpointLib.sol

Description Declaring a top-level structure in a file named after a library makes it harder navigating through code.

Recommendation Move the structure declaration into the library or move it into a separate file.

```
11 struct Checkpoint {
```

CVF-31 FIXED

- **Category** Suboptimal
- **Source** CheckpointLib.sol

Recommendation Binary shift would be more efficient than multiplication here.

```
41 return (uint256(mps) * (FixedPoint96.Q96 << FixedPoint96.RESOLUTION)
    ↪ ) / price;
```

CVF-32 INFO

- **Category** Suboptimal
- **Source** ConstantsLib.sol

Description This denominator is quite unusual.

Recommendation Consider using more conventional denominator, such as 1e6 or 1e18.

```
7 /// @notice we use milli-bips, or one thousandth of a basis point
uint24 constant MPS = 1e7;
```

CVF-33 FIXED

- **Category** Suboptimal
- **Source** ConstantsLib.sol

Recommendation Outer brackets are redundant.

```
10 uint256 constant X7_UPPER_BOUND = (type(uint256).max) / 1e7;
```

CVF-34 INFO

- **Category** Procedural
- **Source** BidLib.sol

Description We didn't review this file.

```
8 import {FixedPointMathLib} from 'solady/utils/FixedPointMathLib.sol'
    ↪ ;
```

CVF-35 INFO

- **Category** Procedural
- **Source** BidLib.sol

Description Declaring a top-level structure in a file named after a library makes it harder navigating through code.

Recommendation Move the structure declaration into the library of move it into a separate file.

Client Comment *Code style related.*

```
10 struct Bid {
```

CVF-36 FIXED

- **Category** Procedural
- **Source** BidLib.sol

Recommendation Brackets are redundant.

```
46 return (bid.amountQ96 * ConstantsLib.MPS) / mpsRemainingInAuction;
```

CVF-37 INFO

- **Category** Procedural
- **Source** CurrencyLibrary.sol

Recommendation The original errors message should be included into these errors as a parameter.

Client Comment *Previously audited as part of v4-core.*

```
15 error NativeTransferFailed();
```

```
18 error ERC20TransferFailed();
```

CVF-38 INFO

- **Category** Procedural
- **Source** AuctionStepLib.sol

Description Declaring a top-level structure in a file named after a library makes it harder navigating through code.

Recommendation Move the structure declaration into the library of move it into a separate file.

Client Comment *Code style related.*

```
4 struct AuctionStep {
```

CVF-39 INFO

- **Category** Suboptimal
- **Source** AuctionStepLib.sol

Description This bitwise “and” operation is redundant, as Solidity tolerates garbage in unused bits of narrow types.

Recommendation Remove redundant operation.

Client Comment *Won’t fix, prefer the explict operation.*

```
26 blockDelta := and(packedValue, 0xFFFFFFFFFF)
```

CVF-40 INFO

- **Category** Bad datatype
- **Source** IDistributionStrategy.sol

Recommendation The type for the "token" argument should be "IERC20".

Client Comment *Won't fix, part of larger system.*

```
19 function initializeDistribution(address token, uint256 amount, bytes
   ↪  calldata configData, bytes32 salt)
```

CVF-41 INFO

- **Category** Suboptimal
- **Source** IERC20Minimal.sol

Description It is unclear why one would prefer this minimal interface versus the standard IERC20.

Recommendation Explain the potential use cases for this interface in the documentation comment.

Client Comment *Won't fix for now.*

```
4 /// @notice Minimal ERC20 interface
  interface IERC20Minimal {
```

CVF-42 FIXED

- **Category** Suboptimal
- **Source** IAuctionStepStorage.sol

Recommendation These errors could be made more useful by adding certain parameters into them.

Client Comment *Fixed, added params to InvalidStepDataMps and InvalidEndBlockGivenStepData.*

```
9  error InvalidEndBlock();

11 error AuctionIsOver();

13 error InvalidAuctionDataLength();

15 error StepBlockDeltaCannotBeZero();

17 error InvalidStepDataMps();

19 error InvalidEndBlockGivenStepData();
```

CVF-43 INFO

- **Category** Bad datatype
- **Source** IAuctionStepStorage.sol

Recommendation The return type should be more specific.

```
30 function pointer() external view returns (address);
```

CVF-44 INFO

- **Category** Bad naming
- **Source** IAuctionStepStorage.sol

Recommendation Events are usually named via nouns, such as “AuctionStep”.

```
39 event AuctionStepRecorded(uint256 indexed startBlock, uint256
   ↪ indexed endBlock, uint24 mps);
```

CVF-45 FIXED

- **Category** Suboptimal
- **Source** IBidStorage.sol

Recommendation This error could be made more useful by adding certain parameters into it.

```
8 error BidIdDoesNotExist();
```

CVF-46 FIXED

- **Category** Documentation
- **Source** ICheckpointStorage.sol

Description The number format for this argument is unclear.
Recommendation Explain in a documentation comment.

```
13 /// @return The current clearing price
```

CVF-47 INFO

- **Category** Suboptimal
- **Source** ITickStorage.sol

Recommendation These errors could be made more useful by adding certain parameters into them.

Client Comment *Not much benefit from adding params.*

```
11 error TotalSupplyIsTooLarge();
```

```
15 error TokenAndCurrencyCannotBeTheSame();
```

```
23 error CannotSweepCurrency();
```

```
25 error CannotSweepTokens();
```

```
27 error NotGraduated();
```

```
29 error RequiredCurrencyRaisedIsTooLarge();
```

CVF-48 FIXED

- **Category** Suboptimal
- **Source** ITokenCurrencyStorage.sol

Recommendation These errors could be made more useful by adding certain parameters into them.

Client Comment *Won't fix, but remove unused error.*

CVF-49 INFO

- **Category** Bad naming
- **Source** ITokenCurrencyStorage.sol

Recommendation Events are usually named via nouns, such as "TokensSweep" or "CurrencySweep".

```
34 event TokensSwept(address indexed tokensRecipient, uint256
   ↪ tokensAmount);
```

```
39 event CurrencySwept(address indexed fundsRecipient, uint256
   ↪ currencyAmount);
```

CVF-50 INFO

- **Category** Procedural
- **Source** IAuction.sol

Description Declaring a top-level structure in a file named after an interface makes it harder navigating through code.

Recommendation Move the structure declaration into the interface or move it into a separate file.

Client Comment *Code style.*

```
16 struct AuctionParameters {
```

CVF-51 INFO

- **Category** Bad datatype
- **Source** IAuction.sol

Recommendation The type for this field should be “Currency”.

```
17 address currency; // token to raise funds in. Use address(0) for ETH
```

CVF-52 INFO

- **Category** Bad datatype
- **Source** IAuction.sol

Recommendation The type for this field should be “IValidationHook”.

Client Comment *Won’t fix*

```
24 address validationHook; // Optional hook called before a bid
```

CVF-53 INFO

- **Category** Suboptimal
- **Source** IAuction.sol

Recommendation These errors could be made more useful by adding certain parameters into them.

Client Comment *Won’t fix.*

CVF-54 INFO

- **Category** Bad naming
- **Source** IAuction.sol

Recommendation Events are usually named via nous, such as “Tokens” or “Bid”.

```
87 event TokensReceived(uint256 totalSupply);
```

```
94 event BidSubmitted(uint256 indexed id, address indexed owner,
   ↪ uint256 price, uint256 amount);
```

```
101 event CheckpointUpdated(
```

```
107 event ClearingPriceUpdated(uint256 indexed blockNumber, uint256
   ↪ clearingPrice);
```

```
114 event BidExited(uint256 indexed bidId, address indexed owner,
   ↪ uint256 tokensFilled, uint256 currencyRefunded);
```

```
120 event TokensClaimed(uint256 indexed bidId, address indexed owner,
   ↪ uint256 tokensFilled);
```

CVF-55 INFO

- **Category** Bad naming
- **Source** IAuctionFactory.sol

Recommendation Event are usually named via nouns, such as “Auction”.

```
16 event AuctionCreated(address indexed auction, address token, uint256
   ↪  amount, bytes configData);
```

CVF-56 INFO

- **Category** Bad datatype
- **Source** IAuctionFactory.sol

Recommendation The type for the “auction” parameter should be “IAuction”.
Client Comment *Won’t fix.*

```
16 event AuctionCreated(address indexed auction, address token, uint256
   ↪  amount, bytes configData);
```

CVF-57 INFO

- **Category** Bad datatype
- **Source** IAuctionFactory.sol

Recommendation The type for the “token” parameter should be “IERC20”.
Client Comment *Won’t fix.*

```
16 event AuctionCreated(address indexed auction, address token, uint256
   ↪  amount, bytes configData);
```

CVF-58 INFO

- **Category** Bad datatype
- **Source** IAuctionFactory.sol

Recommendation The type for the “token” argument should be “IERC20”.
Client Comment *Won’t fix.*

```
25 function getAuctionAddress(address token, uint256 amount, bytes
   ↪ calldata configData, bytes32 salt, address sender)
```

CVF-59 INFO

- **Category** Bad datatype
- **Source** IAuctionFactory.sol

Recommendation The return type should be “IAuction”.
Client Comment *Won’t fix.*

```
28 returns (address);
```

CVF-60 INFO

- **Category** Procedural
- **Source** TickStorage.sol

Description Declaring a top-level structure in a file named after a contract makes it harder navigating through code.
Recommendation Move the structure declaration into the contract or move it into a separate file.
Client Comment *Code style related.*

```
8 struct Tick {
```

CVF-61 INFO

- **Category** Suboptimal
- **Source** TickStorage.sol

Description Using the dollar sign in the name of a storage variable looks weird. Dollar is commonly used for temporary variables.

Recommendation Don't use the dollar sign here. See also: AuctionStepStorage.sol, BidStorage.sol, CheckpointStorage.sol.

Client Comment *Code style related.*

```
17 mapping(uint256 price => Tick) private $_ticks;
```

```
25 address private immutable $_pointer;
```

```
27 uint256 private $_offset;
```

```
29 AuctionStep internal $step;"
30 ␣␣␣␣␣␣
```

```
10 uint256 private $_nextBidId;
```

```
12 mapping(uint256 bidId => Bid bid) private $_bids;"
   ␣␣␣␣␣␣
```

```
26 mapping(uint64 blockNumber => Checkpoint) private $_checkpoints;
```

```
28 uint64 internal $lastCheckpointedBlock;"
   ␣␣␣␣␣␣
```

CVF-62 INFO

- **Category** Procedural
- **Source** TickStorage.sol

Recommendation This check should be performed earlier.

Client Comment *Can't move this earlier because we don't want to require that 'prevPrice' is submitted by the caller if the 'price' is already initialized.*

```
71 if (prevPrice >= price) revert TickPreviousPriceInvalid();
```

CVF-63 INFO

- **Category** Procedural
- **Source** AuctionStepStorage.sol

Description We didn't review this file.

```
7 import {SSTORE2} from 'solady/utils/SSTORE2.sol';
```

CVF-64 INFO

- **Category** Suboptimal
- **Source** PermitSingleForwarder.sol

Recommendation This constant is not used and should be removed, as hardcoding mainnet addresses is a bad practice in general.

Client Comment *The constant is used (initialized in Auction.sol:constructor) and this is a common pattern we use across repos, Permit2 has a constant address*

```
10 address public constant PERMIT2 = 0
   ↪ x000000000022D473030F116dDEE9F6B43aC78BA3;
```

CVF-65 INFO

- **Category** Procedural
- **Source** TokenCurrencyStorage.sol

Description We didn't review this file.

```
12 import {FixedPointMathLib} from 'solady/utils/FixedPointMathLib.sol'
   ↪ ;
```

CVF-66 INFO

- **Category** Procedural
- **Source** TokenCurrencyStorage.sol

Description UPPER_CASE identifiers are commonly used for constants, not for immutable variables.

Recommendation Use camelCase instead.

Client Comment *Code style related.*

```
22 Currency internal immutable CURRENCY;
```

```
24 IERC20Minimal internal immutable TOKEN;
```

```
26 uint128 internal immutable TOTAL_SUPPLY;
```

```
28 uint256 internal immutable TOTAL_SUPPLY_Q96;
```

```
30 address internal immutable TOKENS_RECIPIENT;
```

```
32 address internal immutable FUNDS_RECIPIENT;
```

```
34 uint256 internal immutable REQUIRED_CURRENCY_RAISED_Q96;
```

CVF-67 INFO

- **Category** Bad datatype
- **Source** TokenCurrencyStorage.sol

Recommendation The type for this argument should be “IERC20”.

```
42 address _token,
```

CVF-68 INFO

- **Category** Bad datatype
- **Source** TokenCurrencyStorage.sol

Recommendation The type for this argument should be “Currency”.

```
43 address _currency,
```

CVF-69 FIXED

- **Category** Procedural
- **Source** TokenCurrencyStorage.sol

Recommendation This check should be performed earlier.
Client Comment *This error is removed as it can't be hit.*

```
56 if (_requiredCurrencyRaised * FixedPoint96.Q96 > ConstantsLib.
   ↪ X7_UPPER_BOUND) {
    revert RequiredCurrencyRaisedIsTooLarge();
```

CVF-70 FIXED

- **Category** Suboptimal
- **Source** TokenCurrencyStorage.sol

Description The value "_requiredCurrencyRaised * FixedPoint96.Q96 is calculated twice.
Recommendation Calculate once and reuse.
Client Comment *The error is removed so only calcualted once.*

```
56 if (_requiredCurrencyRaised * FixedPoint96.Q96 > ConstantsLib.
   ↪ X7_UPPER_BOUND) {
    revert RequiredCurrencyRaisedIsTooLarge();
```

```
59 REQUIRED_CURRENCY_RAISED_Q96 = _requiredCurrencyRaised *
   ↪ FixedPoint96.Q96;
```

CVF-71 FIXED

- **Category** Procedural
- **Source** TokenCurrencyStorage.sol

Recommendation These checks should be performed earlier.

```
61 if (_token == address(0)) revert TokenIsAddressZero();
   if (_token == address(_currency)) revert
      ↪ TokenAndCurrencyCannotBeTheSame();
   if (TOKENS_RECIPIENT == address(0)) revert TokensRecipientIsZero();
   if (FUNDS_RECIPIENT == address(0)) revert FundsRecipientIsZero();
```

CVF-72 INFO

- **Category** Suboptimal
- **Source** TokenCurrencyStorage.sol

Recommendation These checks are redundant as it is anyway possible to pass dead addresses.

Client Comment *Won't fix, these are extra safeguards for uninitialized values.*

```
61 if (_token == address(0)) revert TokenIsAddressZero();
   if (_token == address(_currency)) revert
      ↪ TokenAndCurrencyCannotBeTheSame();
   if (TOKENS_RECIPIENT == address(0)) revert TokensRecipientIsZero();
   if (FUNDS_RECIPIENT == address(0)) revert FundsRecipientIsZero();
```

CVF-73 INFO

- **Category** Bad datatype
- **Source** AuctionFactory.sol

Recommendation The type for the "token" argument should be "IERC20".

```
16 function initializeDistribution(address token, uint256 amount, bytes
      ↪  calldata configData, bytes32 salt)
```

```
36 function getAuctionAddress(address token, uint256 amount, bytes
      ↪ calldata configData, bytes32 salt, address sender)
```

CVF-74 INFO

- **Category** Bad datatype
- **Source** AuctionFactory.sol

Recommendation The return type should be "IAuction".

```
39 returns (address)
```

CVF-75 INFO

- **Category** Procedural
- **Source** Auction.sol

Description We didn't review these files.

```
24 import {FixedPointMathLib} from 'solady/utils/FixedPointMathLib.sol'
      ↪ ;
   import {SafeCastLib} from 'solady/utils/SafeCastLib.sol';
   import {SafeTransferLib} from 'solady/utils/SafeTransferLib.sol';
```

CVF-76 INFO

- **Category** Bad datatype
- **Source** Auction.sol

Recommendation The type for the "_token" address should be "IERC20".
Client Comment *Won't fix.*

```
67 constructor(address _token, uint128 _totalSupply, AuctionParameters
   ↪ memory _parameters)
```

CVF-77 INFO

- **Category** Suboptimal
- **Source** Auction.sol

Description Here the value is scaled twice, which is inefficient.
Recommendation Refactor the code to scale once.
Client Comment *Prefer explicitness.*

```
144 return $currencyRaisedQ96_X7.scaleDownToUint256() >> FixedPoint96.
    ↪ RESOLUTION;
```

CVF-78 FIXED

- **Category** Readability
- **Source** Auction.sol

Recommendation Should be "else return".

```
406 return _unsafeCheckpoint(uint64(block.number));
```

CVF-79 INFO

- **Category** Suboptimal
- **Source** Auction.sol

Description The ame condition is checked twice.
Recommendation Refactor the code to avoid checking the same condition twice.
Client Comment *Won't fix.*

```
526     if (upperCheckpoint.clearingPrice != bidMaxPrice) {
```

```
548 if (upperCheckpoint.clearingPrice == bidMaxPrice) {
```

CVF-80 FIXED

- **Category** Documentation
- **Source** Auction.sol

Description These diagrams are confusing.
Recommendation Elaborate more in the comment.

```
532 * Account for partially filled checkpoints
    *
    *                 <-- fully filled ->  <- partially filled
       ↪ ---------->  INACTIVE
    *                | ----------------- | -------- |
       ↪ ------------------- | ------ |
    *                ^                   ^          ^
       ↪                      ^        ^
    *              start       lastFullyFilled   lastFullyFilled.next
       ↪    upper    outbid
    *
    * Instantly partial fill case:
540 *
    *                <- partially filled ----------------------------->
       ↪  INACTIVE
    *                | ----------------- | -------------------------- |
       ↪  ------ |
    *                ^                   ^                            ^
       ↪         ^
    *               start          lastFullyFilled.next
       ↪ upper    outbid
    *           lastFullyFilled
```

CVF-81 INFO

- **Category** Procedural
- **Source** CheckpointStorage.sol

Description We didn't review this file.

```
11 import {FixedPointMathLib} from 'solady/utils/FixedPointMathLib.sol'
      ↪ ;
```

ABDK
Consulting

About us

Established in 2016, is a leading service provider in the space of blockchain development and audit. It has contributed to numerous blockchain projects, and co-authored some widely known blockchain primitives like Poseidon hash function.

The ABDK Audit Team, led by Mikhail Vladimirov and Dmitry Khovratovich, has conducted over 300 audits of blockchain projects in Solidity, Rust, Circom, C++, JavaScript, and other languages.

Contact

Email

dmitry@abdkconsulting.com

Website

abdk.consulting

Twitter

twitter.com/ABDKconsulting

LinkedIn

linkedin.com/company/abdk-consulting